

Course ID: SW 302

User Interface (UI) Development

Dr. Mohamed Sami Rakha

Lecture#12

Redux & Testing

Lecture Outline

Dr. Mohamed Sami Rakha

Redux, Custom Hooks, Data Visualization & Testing

Hooks: Easy Mental Model

Hook	Remembers
useState	Data
useEffect	Side effects
useContext	Shared state
useReducer	Update logic
useCallback	Functions
useMemo	Values



Rakha



One more Hook: useRef

One more Hook: useRef

`useRef` stores a value that doesn't cause a re-render when it changes. `useRef` lets you **store a value or DOM element without causing a re-render.**

What problems does useRef solve?

- Problem 1: Accessing DOM elements safely.
- Problem 2: Keeping values between renders.
- Problem 3: Storing mutable values (timers, intervals)

Problem 1: Accessing DOM elements safely.

```
import { useRef } from "react";

function InputFocus() {
  const inputRef = useRef(null);

  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>
        Focus Input
      </button>
    </>
  );
}
```

Why This Is Better Than useState

```
const [input, setInput] = useState(null);
```

Why this fails:

- Changing state: re-render
- You don't want a re-render just to focus
- State is for UI data, not DOM references

[1] useRef(null): React creates an object:

`const inputRef = useRef(null);` This object **persists between renders**

Dr. Mohamed Sami Rakha { current: null }

[2] ref={inputRef}: React connects the real DOM element to:

```
<input ref={inputRef} />
```

After render: `inputRef.current === <input DOM element>`

[3] Button Click `inputRef.current.focus();`

Calls the browser's focus method

- No re-render
- No state update
- Instant UI effect

Why not `document.querySelector`?

- Breaks React rules
- Unsafe in component lifecycles
- Hard to test

`useRef` is the **React-approved way**.

Problem 2: Keeping values between renders (without re-rendering)

Dr. Mohamed Sami Rakha

```
import { useRef, useEffect } from "react";

function RenderCounter() {
  const renders = useRef(0);

  useEffect(() => {
    renders.current++;
  });

  return <p>Renders: {renders.current}</p>;
}
```

Sometimes you need memory, but **not UI updates**.

Examples:

- Previous value
- Timer ID
- Click count
- Is first render?

Using **useState** would re-render, unnecessary.

Why useRef here?

- Value persists
- No re-render loop
- State would cause infinite renders

Problem 3: Storing mutable values (timers, intervals)

```
import { useRef, useEffect, useState } from "react";

function Timer() {
  const intervalRef = useRef(null);
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    intervalRef.current = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);

    return () => clearInterval(intervalRef.current);
  }, []);

  return <p>Seconds: {seconds}</p>;
}
```

useRef stores a mutable value that persists across re-renders without causing re-renders

What Happens Step by Step

1. setInterval is created
2. After 1000 ms, the callback runs
3. setSeconds updates the state
4. React re-renders the component
5. Every 1000 ms, it repeats

Why not a local variable?

Local variables in a React function component are reset on every re-render

Why not just store the interval ID in state?
Causes re-render when interval ID changes

- **useRef** does NOT prevent re-renders, it allows us to safely store the interval ID across re-renders so we can clear it later.
- The code **runs repeatedly every 1 second** and is cleared **only when the component unmounts** (or when the effect cleans up).
- The clearInterval here is needed to clear up the running loop of setInterval using the referred id , **otherwise, timer runs forever and causes bugs**

Note: setSeconds(s => s + 1) should work fine, why? The "s" here is just a parameter name chosen by the developer.

- **s** means "the previous value of the state"
- React passes it in automatically
- You can name it anything you want

Important: The value passed to s is the previous value of the seconds state. Not because of the letter "s", but because setSeconds is the updater for the seconds state.

Why needed? guarantees that React uses the latest state value, even when updates are asynchronous.

Custom Hooks

What is a Custom Hook?

A JavaScript function that **starts with** use and reuses stateful logic using React hooks. Custom Hooks allow us to **extract and reuse logic** between components.

The problem: duplicated logic

```
function CounterA() {  
  const [count, setCount] = useState(0);  
  const increment = () => setCount(c => c + 1);  
}  
  
function CounterB() {  
  const [count, setCount] = useState(0);  
  const increment = () => setCount(c => c + 1);  
}
```

- ✓ It is not a component
- ✓ It does not return JSX
- ✓ It returns logic, state, or functions

- ✗ Repeated logic
- ✗ Hard to maintain
- ✗ Violates DRY principle

Custom Hooks follow **the same rules as React hooks**:

1. Must start with **use**
2. Can only call hooks:
 - At the top level
 - Inside components or other hooks
3. Cannot be conditional
4. Cannot be called inside loops

What is a Custom Hook?

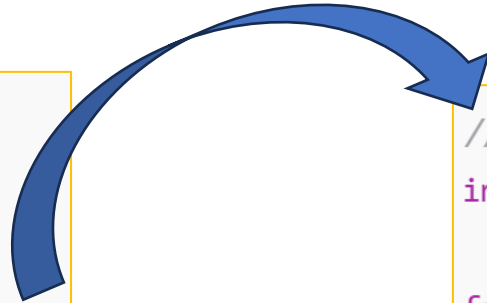
Now both components reuse the same logic.

```
// src/hooks/useCounter.js
import { useState } from "react";

export function useCounter(initial = 0) {
  const [count, setCount] = useState(initial);

  const increment = () => setCount(c => c + 1);
  const decrement = () => setCount(c => c - 1);

  return { count, increment, decrement };
}
```



Dr. Mohamed Sami Rakha

```
// CounterA.jsx
import { useCounter } from "../useCounter";

function CounterA() {
  const { count, increment, decrement } = useCounter(0);

  return (
    <div>
      <h3>Counter A</h3>
      <p>{count}</p>
      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
    </div>
  );
}

export default CounterA;
```

Note: `setCount(c => c + 1)` works correctly as it uses the **functional updater pattern** where `setCount` is tied to the count state. React guarantees that the function form receives the **previous value of count**

Custom Hooks with Side Effects

Example: useDocumentTitle

Dr. Mohamed Sami Rakha

```
import { useEffect } from "react";

export function useDocumentTitle(title) {
  useEffect(() => {
    document.title = title;
  }, [title]);
}
```



```
useDocumentTitle("Dashboard");
```

- ✓ Cleaner components
- ✓ Side effects isolated

Custom Hooks with useRef

Example: useAutoFocus

Dr. Mohamed Sami Rakha

```
import { useRef, useEffect } from "react";

export function useAutoFocus() {
  const ref = useRef(null);

  useEffect(() => {
    ref.current?.focus();
  }, []);

  return ref;
}
```

Usage:

```
const inputRef = useAutoFocus();


```

- ✓ Encapsulates imperative logic
- ✓ Keeps components declarative

Redux

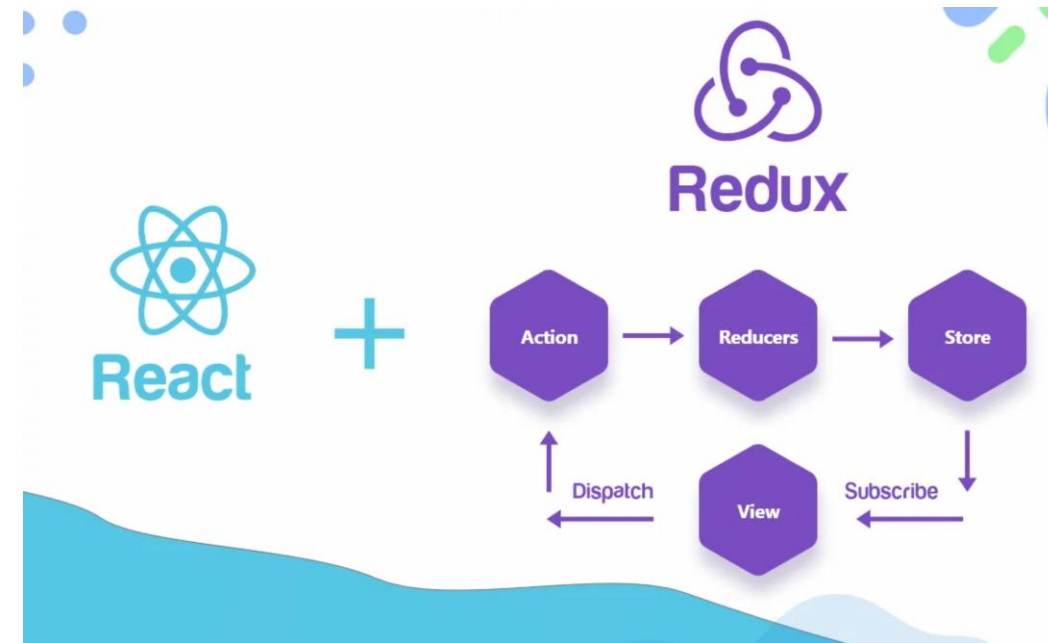
What is Redux?

Redux is a **state management library** for JavaScript applications, commonly used with React. It helps you **manage and centralize the state** of your app in a predictable way. Instead of having state scattered across many components, Redux keeps all the state in a **single “store”**, and components can access it or update it through **actions** and **reducers**.

Key points about Redux:

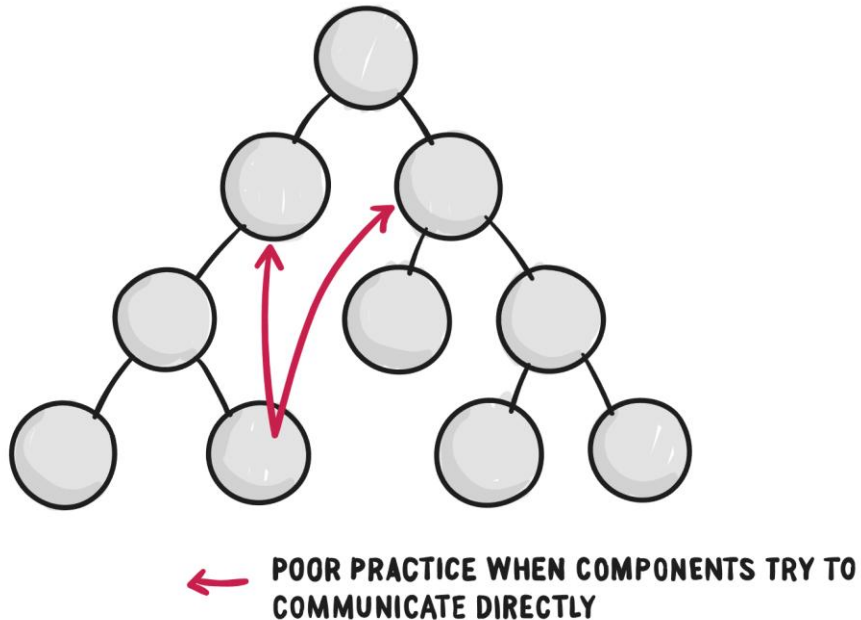
- 1. Single Source of Truth:** All your app's state lives in one central place called the **store**.
- 2. Predictable State Updates:** State can only be changed by **dispatching actions**, which are handled by **reducers**.
- 3. Traceable Changes:** Every state change is explicit, making debugging easier.
- 4. Works with React and Other Libraries:** While popular with React, Redux can be used with any JS framework.

Think of Redux like a **global notebook** for your app. Components write to it (actions), a set of rules decides how it's updated (reducers), and all components can read from it (store).



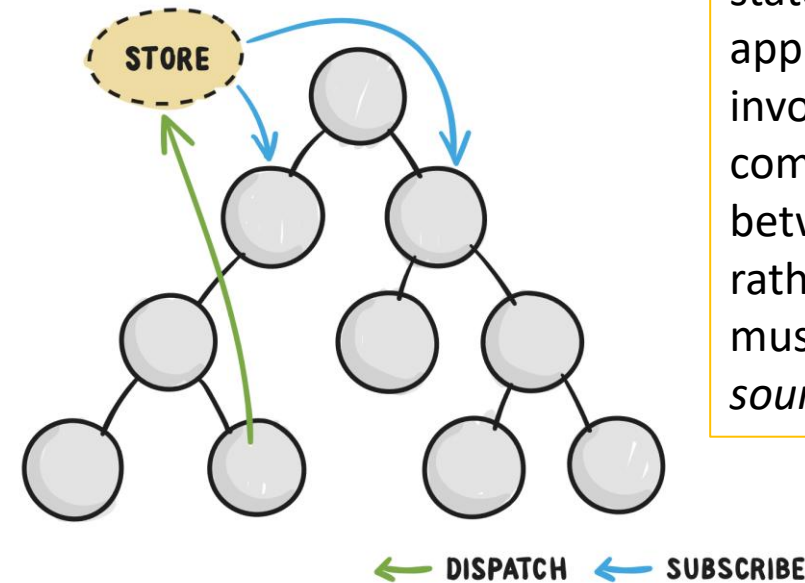
Unidirectional data flow

Normal, data flow through components in “unidirectional data flow”, data flows in one direction from parent to child. With this characteristic, it’s not obvious how **two non parent-child components** would communicate in React:



React doesn’t recommend direct component-to-component communication this way. Even if it did have features to support this approach, it’s considered poor practice by many because direct component-to-component communication is error prone and leads to [spaghetti code](#), an old term for code that is hard to follow.

This is where Redux comes in handy. Redux offers a solution of storing all your application state in one place, called a “store”. Components then “dispatch” state changes to the store, not directly to other components. The components that need to be aware of state changes can “subscribe” to the store:



The store can be thought of as a “middleman” for all state changes in the application. With Redux involved, components don’t communicate directly between each other, but rather all state changes must go through the *single source of truth*, the store.

Counter App with Redux

Concept

- App shows a number (state)
- Two buttons: **Increment / Decrement**
- Clicking buttons **dispatches actions** that update the state

[1] Redux Setup

```
npm install react-redux redux
```

[3] Create the Redux Store

src/store/store.js

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../counterSlice";
```

The `counterReducer` is just the name we give it when importing

```
export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

Store holds the global state (`counter.value`).

[2] Create the Counter Slice (Reducer + Actions)

src/store/counterSlice.js

```
import { createSlice } from "@reduxjs/toolkit";
```

```
const counterSlice = createSlice({
  name: "counter",
  initialState: {
    value: 0
  },
  reducers: {
    increment(state) {
      state.value += 1;
    },
    decrement(state) {
      state.value -= 1;
    },
    reset(state) {
      state.value = 0;
    }
  }
});
```

```
export const { increment, decrement, reset } = counterSlice.actions;
export default counterSlice.reducer;
```

In **Redux Toolkit**, when you create a slice using `createSlice()`, the slice automatically **generates a reducer** for you. That reducer is **exported as the default** from the slice file.

`createSlice()` generates:

1. **Actions:** `increment, decrement, reset`
2. **Reducer:** `counterSlice.reducer`

Counter App with Redux

4. Connect Redux to React

src/main.jsx or index.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import { store } from "../store/store";
import App from "../App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Provider makes Redux state available to all components

6. Data Flow

Button Click -> dispatch(increment()) -> Reducer updates state
-> Redux Store updates -> useSelector detects change
-> Component re-renders

5. Use Redux in a Counter Component

src/components/CounterRedux.jsx

```
import { useDispatch, useSelector } from "react-redux";
import { increment, decrement, reset } from "../store/counterSlice";

function CounterRedux() {
  const value = useSelector((state) => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <h2>Redux Counter</h2>
      <p>Value: {value}</p>

      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() => dispatch(reset())}>Reset</button>
    </div>
  );
}

export default CounterRedux;
```

Recharts

Recharts in React

Recharts is a charting library built specifically for React. It allows developers to create charts using React components instead of manually drawing graphics.

Why Use Recharts in React?

- Designed for React applications
- Uses a component-based and declarative approach
- Automatically updates when React state or Redux state changes
- Simple API with minimal configuration

How Recharts Works in React

- Charts are React components (LineChart, BarChart, PieChart, etc.)
- Data is passed as props
- When data changes, React re-renders the chart
- No direct DOM manipulation is required

Basic Usage Flow

1. Install Recharts
2. Import chart components
3. Prepare data as an array of objects
4. Render the chart inside a React component
5. Connect the chart to the state or Redux

```
npm install recharts
```

How Recharts Integrates with React State

Recharts charts are **controlled by data**, just like any other React component.

[1] Data Structure Requirement

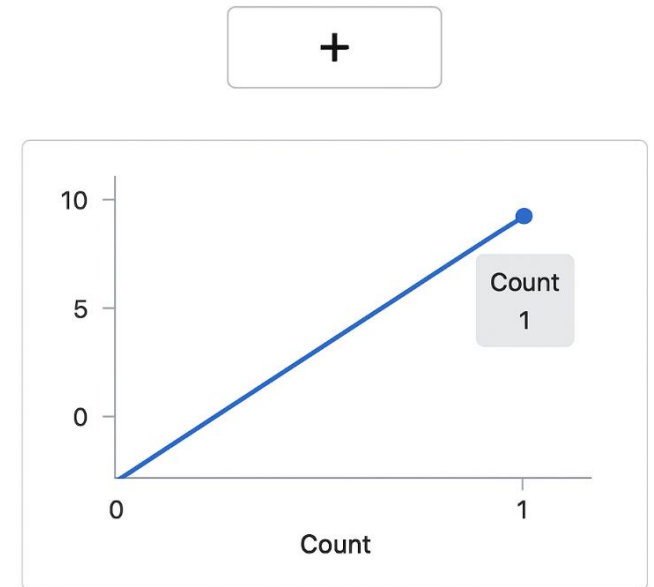
Recharts expects data in this format:

```
const data = [  
  { name: "A", value: 10 },  
  { name: "B", value: 20 },  
  { name: "C", value: 15 }  
];
```

- Each object represents one point or bar
- Keys are mapped using dataKey

Example: Chart Connected to State

```
import { LineChart, Line, XAxis, YAxis, Tooltip } from "recharts";  
import { useState } from "react";  
  
function SimpleChart() {  
  const [count, setCount] = useState(0);  
  
  const data = [{ name: "Count", value: count }];  
  
  return (  
    <>  
      <button onClick={() => setCount(count + 1)}>+</button>  
  
      <LineChart width={300} height={200} data={data}>  
        <XAxis dataKey="name" />  
        <YAxis />  
        <Tooltip />  
        <Line dataKey="value" />  
      </LineChart>  
    </>  
  );  
}
```



Example: Creating the Product Price Bar Chart

In this component, we'll fetch product data from the API and visualize it with a bar chart.

```
// src/components/ProductChart.jsx
import React from 'react';
import axios from 'axios';
import { useState, useEffect } from 'react';
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer } from 'recharts';

export default function ProductChart() {
  const [products, setProducts] = useState([]);

  const fetchData = async () => {
    try {
      const response = await axios.get('https://fakestoreapi.com/products');
      setProducts(response.data);
    } catch (err) {
      console.log(err);
    }
  };

  useEffect(() => {
    fetchData();
  }, []);

  // Prepare data for chart
  const chartData = products.map(item => ({
    name: item.id,
    price: item.price,
  }));

  return (
    <>
      <div className='product'>
        <h3 className='product-heading'>PRODUCT PRICE BAR CHART</h3>
        <ResponsiveContainer width='95%' height={450}>
          <BarChart
            data={chartData}
            margin={{
              top: 5,
              right: 30,
              left: 20,
              bottom: 20,
            }}
          >
            <CartesianGrid strokeDasharray="3 3" />
            <XAxis dataKey="name" label={{ value: "Product ID", position: "bottom" }} />
            <YAxis label={{ value: "Price", angle: -90, position: "insideLeft", }} />
            <Tooltip />
            <Bar dataKey="price" fill="#eca1ac" />
          </BarChart>
        </ResponsiveContainer>
      </div>
    </>
  );
}
```

Explanation

ResponsiveContainer: Makes the chart responsive by setting its width to 95% and height to 450px. **BarChart:** The main component that renders the bar chart.

CartesianGrid: Adds a background grid for readability.

XAxis & YAxis: Set up the labels for product ID and price.

Tooltip: Shows data when hovering over the bars.

Bar: Renders the bars, with each bar representing a product price.



Example: Creating the Product Categories Pie Chart

In this component, we'll fetch product data, count the products in each category, and visualize them using a pie chart.

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { PieChart, Pie, Tooltip, Cell, ResponsiveContainer, Legend } from 'recharts';

export default function CategoryChart() {
  const [chartData, setChartData] = useState([]);

  const fetchData = async () => {
    try {
      const response = await axios.get('https://fakestoreapi.com/products');
      categoryCount(response.data);
    } catch (err) {
      console.log(err);
    }
  };

  const categoryCount = (products) => {
    const count = {};
    products.forEach((product) => {
      if (count[product.category]) {
        count[product.category]++;
      } else {
        count[product.category] = 1;
      }
    });
  };

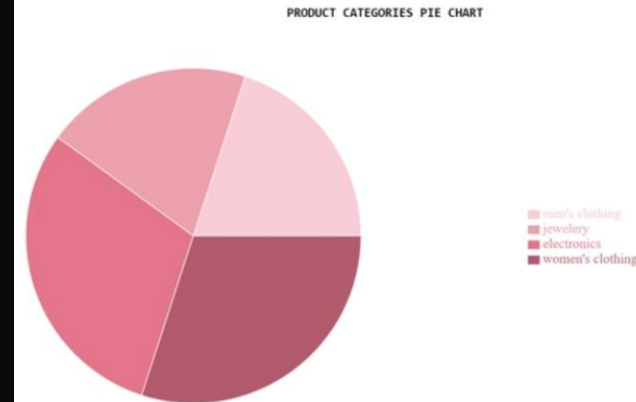
  const chartData = Object.keys(count).map((category) => ({
    name: category,
    value: count[category],
  }));

  setChartData(chartData);

  useEffect(() => {
    fetchData();
  }, []);

  const COLORS = ['#f9cdd4', '#eca1ac', '#e27589', '#b25b6e', '#7c3042'];

  return (
    <div>
      <h3 className='product-heading'>PRODUCT CATEGORIES PIE CHART</h3>
      <ResponsiveContainer width='80%' height={450}>
        <PieChart>
          <Pie
            data={chartData}
            cx="45%"
            cy="50%"
            outerRadius={200}
            fill="#8884d8"
            dataKey="value"
          />
          {chartData.map((entry, index) => (
            <Cell key={`cell-${index}`} fill={COLORS[index % COL
              ))}
          </Pie>
          <Tooltip />
          <Legend layout="vertical" align="right" verticalAlign="middl
        </PieChart>
      </ResponsiveContainer>
    </div>
  );
}
```



Explanation

categoryCount Method: Counts the number of products in each category and formats the result for display in the pie chart.

PieChart & Pie: The main components that render the pie chart.

Cell: Adds color to each pie slice, defined by the COLORS array.

cx, cy, and outerRadius: Position and size the pie chart within the container.

Rendering Components in App.js

To see your charts in action, you need to render these components in App.js.

Update src/App.js as follows:

```
// src/App.js
import React from 'react'
import ProductChart from './components/ProductChart'
import './app.css'
import CategoryChart from './components/CategoryChart'

export default function App() {
  return (
    <div className='app'>
      <ProductChart/>
      <CategoryChart/>
    </div>
  )
}
```

With this in place, you should see both the bar chart and pie chart on your screen!

React Testing with Jest

React Testing Jest

Testing is crucial in React applications to ensure components behave correctly and prevent regressions. Jest is the most widely used testing framework for React. It works with the React Testing Library to simulate component rendering and user interactions.

Simple Example: Testing a Greeting Component

```
// Hello.js
function Hello({ name }) {
  return <h1>Hello, {name}!</h1>;
}
export default Hello;

// Hello.test.js
import { render, screen } from '@testing-library/react';
import Hello from './Hello';

test('renders greeting', () => {
  render(<Hello name="Mo" />);
  expect(screen.getByText('Hello, Mo!')).toBeInTheDocument();
});
```

Why Test React Components:

- Verify UI renders correctly.
 - Test user interactions (clicks, inputs, form submissions).
 - Ensure Redux or context state changes propagate correctly.
 - Detect regressions early during development.
- This is a unit test for a simple component.
 - render() mounts the component in a virtual DOM.
 - screen.getByText() queries the DOM for text.
 - expect(...).toBeInTheDocument() checks if it exists.

Example: Unit Test for a Counter Button

Component: CounterButton.js

```
import { useState } from 'react';

function CounterButton() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>
        Clicked {count} times
      </button>
    </div>
  );
}

export default CounterButton;
```

Test File: CounterButton.test.js

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import CounterButton from './CounterButton';

test('increments count when button is clicked', () => {
  render(<CounterButton />);

  const button = screen.getByText('Clicked 0 times');

  // Simulate a click
  userEvent.click(button);

  // Check if the count updated
  expect(screen.getByText('Clicked 1 times')).toBeInTheDocument();
});
```

- **render()** mounts the component in a virtual DOM.
- **screen.getByText()** finds elements by text content.
- **userEvent.click()** simulates user interaction.
- **expect(...).toBeInTheDocument()** asserts that the new text appears after the click.

Example: Integration Test for a Todo App

Component 1: TodoList.js

```
import TodoItem from './TodoItem';

function TodoList({ todos }) {
  return (
    <ul>
      {todos.map((todo) => (
        <TodoItem key={todo.id} todo={todo} />
      ))}
    </ul>
  );
}

export default TodoList;
```

Component 2: TodoItem.js

```
function TodoItem({ todo }) {
  return <li>{todo.text}</li>;
}

export default TodoItem;
```

Integration Test: TodoList.test.js

```
import { render, screen } from '@testing-library/react';
import TodoList from './TodoList';

test('renders all todo items', () => {
  const todos = [
    { id: 1, text: 'Learn React' },
    { id: 2, text: 'Write Tests' },
  ];

  render(<TodoList todos={todos} />);

  // Check if both items are rendered
  expect(screen.getByText('Learn React')).toBeInTheDocument();
  expect(screen.getByText('Write Tests')).toBeInTheDocument();
});
```

Key Points:

- The test ensures that **TodoList** correctly renders multiple **TodoItem** components.
- This is an integration test because it tests the interaction between **TodoList** and **TodoItem**.
- You're testing the behavior of combined components, not individual implementation details.

End-to-End (E2E) testing

Playwright for E2E Testing

Playwright is a modern E2E testing framework that lets you simulate real user interactions in a browser. The goal is to test full user flows in a real browser environment. It Chromium, Firefox, WebKit (cross-browser testing).

Dr. Mohamed Sami Rakha

Use Case: Testing login flows, navigation, form submissions, and UI workflows.

```
import { test, expect } from '@playwright/test';

test('login works', async ({ page }) => {
  await page.goto('https://example.com/login');
  await page.fill('#username', 'mo');
  await page.fill('#password', 'password123');
  await page.click('button[type="submit"]');
  await expect(page.locator('text=Welcome, Mo')).toBeVisible();
});
```

Key Points:

- Simulates **real user behavior**, not just component rendering.
- Complements **unit and integration tests** written with Jest.
- Ideal for **critical flows** where you need high confidence in your UI.

Important Features:

- Headless or headed mode (run tests without UI)
- Auto-waiting for elements to appear
- Screenshots and video recording of tests
- Network request interception and mocking

React ContactForm Component

```
function ContactForm() {  
  const [formData, setFormData] = useState({  
    name: '',  
    email: '',  
    message: '',  
  });  
  const [submitted, setSubmitted] = useState(false);  
  
  const handleChange = (e) => {  
    setFormData({ ...formData, [e.target.name]: e.target.value });  
  };  
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    // Normally, here you would send data to server via API  
    setSubmitted(true);  
  };  
  
  return (  
    <div>  
      {submitted ? (  
        <p className="success-message">Thank you for your message!</p>  
      ) : (  
        <form onSubmit={handleSubmit}>  
          <div>  
            <label>Name:</label>  
            <input  
              id="name"  
              name="name"  
              type="text"  
              value={formData.name}  
              onChange={handleChange}  
            />  
          </div>  
          <div>  
            <label>Email:</label>  
            <input  
              id="email"  
              name="email"  
              type="email"  
              value={formData.email}  
              onChange={handleChange}  
            />  
          </div>  
          <div>  
            <label>Message:</label>  
            <textarea  
              id="message"  
              name="message"  
              value={formData.message}  
              onChange={handleChange}  
            />  
          </div>  
          <button type="submit">Submit</button>  
        </form>  
      )}  
    </div>  
  );  
}
```

```
    id="email"  
    name="email"  
    type="email"  
    value={formData.email}  
    onChange={handleChange}  
  />  
</div>  
<div>  
  <label>Message:</label>  
  <textarea  
    id="message"  
    name="message"  
    value={formData.message}  
    onChange={handleChange}  
  />  
</div>  
  <button type="submit">Submit</button>  
</form>  
  )}  
</div>  
);  
}
```

export default ContactForm;

Example: E2E Test with Playwright – Contact Form

Test that a contact form submits successfully and shows a success message.

```
import { test, expect } from '@playwright/test';

test('contact form submission', async ({ page }) => {
  // Navigate to the contact page
  await page.goto('https://example.com/contact');

  // Fill out the form
  await page.fill('#name', 'Mo Rakha');
  await page.fill('#email', 'mo@example.com');
  await page.fill('#message', 'Hello! This is a test. ');

  // Submit the form
  await page.click('button[type="submit"]');

  // Verify the success message appears
  await expect(page.locator('.success-message')).toHaveText('Thank you for your message!');
});
```

Dr. Mohamed Sami Rakha

- **Full user flow:** Navigate → Fill → Submit → Verify.
- Tests **real DOM interactions**, not just component output.
- Can run in **headless mode** for CI/CD pipelines.
- Complements **unit and integration tests**, giving full confidence in app behavior.

Dr. Mohamed Sami Rakha

Thank You...